

SQL – pokročilejší DML

Příklad:

Uvažujme lékařskou praxi skupiny lékařů s relační databází se třemi relacemi o schématech:

LÉKAŘ(Č_LICENCE, JMÉNO_L, SPECIALIZACE)

PACIENT(Č_PACIENTA, JMÉNO_P, ADRESA, TELEFON, D_NAROZENÍ)

NÁVŠTĚVA(Č_LICENCE, Č_PACIENTA, TYP, DATUM, DIAGNÓZA, CENA)

kde TYP znamená typ návštěvy (domů na zavolání, do ordinace, do nemocnice apod.). Napište v relační algebře tyto požadavky:

Jména a adresy pacientů, kteří nebyli vůbec ošetřeni.

Jména a adresy pacientů, kteří byli vyšetřováni pouze Dr. Čermákem.

Čísla licencí a jména lékařů, kteří nemají žádný záznam o léčbě (nikoho neošetřili).

Která specializace neměla dosud nic na práci (tj. žádný lékař této specializace nikoho neošetřil).

Čísla pacientů, kteří nebyli ošetřeni některým (tj. aspoň jedním) z lékařů.

Jména a adresy pacientů, kteří byli ošetřeni všemi lékaři.

Zapište v SQL následující dotazy, sledujte korespondenci SQL s relační algebrou (viz předchozí cvičení):

Seznam všech registrovaných pacientů.

Seznam všech specializací lékařů.

Seznam všech orthopédů.

Jména všech orthopédů.

Seznam všech pacientů narozených před r. 1920.

Licence všech lékařů, které navštívila Marie Nová.

Jména všech lékařů, kteří byli na návštěvě domů na zavolání.

Seznam všech navštívených pacientů.

Jména a adresy všech pacientů, kteří byli vyšetřováni Dr. Lomem 23.4.1992.

Jména a adresy všech pacientů, u kterých byla určena diagnóza HIV-positivní.

Nalezni jména a specializace všech lékařů, kteří určili diagnózu vřed na dvanácterníku.

Jména a adresy pacientů, kteří nebyli vůbec ošetřeni.

Jména a adresy pacientů, kteří byli vyšetřováni pouze Dr. Čermákem.

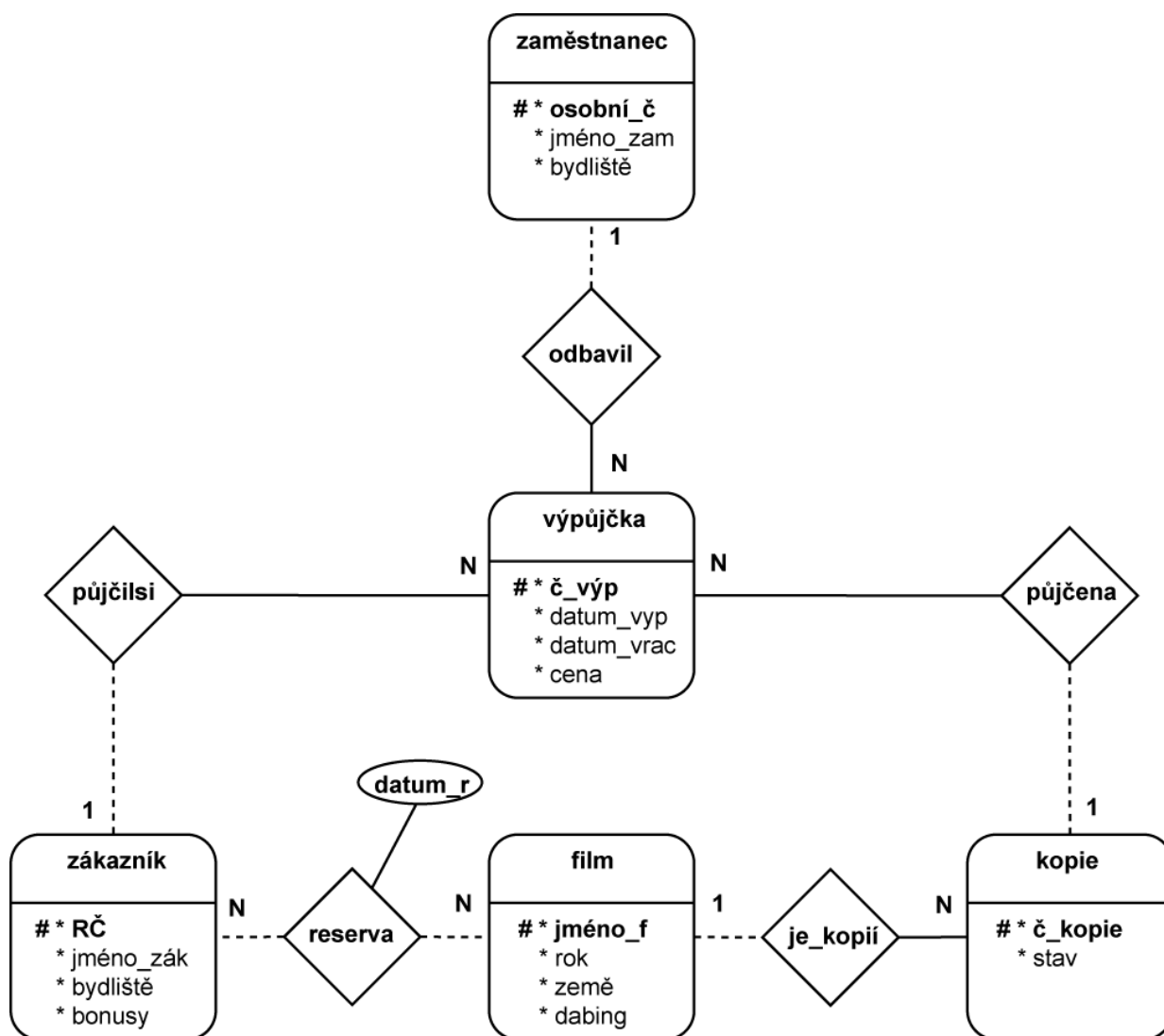
Čísla licencí a jména lékařů, kteří nemají žádný záznam o léčbě.

Která specializace neměla dosud nic na práci.

Čísla pacientů, kteří nebyli ošetřeni aspoň jedním z lékařů.

Jména a adresy pacientů, kteří byli ošetřeni všemi lékaři.

Příklad:



Transformujte do tabulek model dle obrázku, tabulky запиšte v lineární formě a v SQL.

řešení: (podtrženě – primární klíč, kurzíva – cizí klíč)

Zaměstnanec(osobní_č, jméno_zam, bydliště)

Výpůjčka(č_výp, datum_vyp, datum_vrac, cena, *osobní_č*, *rc*, *č_kopie*)

Zákazník(rc, jméno_zák, bydliště, bonusy)

Film(jméno_f, rok, země, dabing)

Kopie(č_kopie, stav, *jméno_f*)

Reservace(rc, *jméno_f*, datum)

Vzhledem k poměrně přímočaré transformaci do SQL uvádím pouze vybrané tabulky jako ilustraci.

```

CREATE TABLE Vypujcka (
    c_vyp      SERIAL      PRIMARY KEY,
    datum_vyp DATE        NOT NULL,
    datum_vrac DATE        CHECK(datum_vrac>=datum_vyp),
    cena       DECIMAL(10,2) CHECK(cena>0),
    osobni_c   INT         NOT NULL REFERENCES Zamestnanec(osobni_c),
    rc         CHAR(10)    NOT NULL REFERENCES Zakaznik(rc),
    c_kopie    INT         NOT NULL REFERENCES Kopie(c_kopie));
  
```

```
CREATE TABLE Reservace (
    rc          INT          REFERENCES Zakaznik(rc),
    jmeno_f     VARCHAR(100) REFERENCES Film(jmeno_f),
    datum      DATE,
    PRIMARY KEY(rc, jmeno_f, datum));
```

Proveďte v SQL níže uvedené dotazy. Poznámka: mnoho dotazů lze zapsat několika způsoby, v některých případech tedy uvádím více možností řešení.

vypiš jména zákazníků s jejich adresami (pozn.: co duplicity?), v SQL je seřaďte podle jména

řešení:

```
SELECT jmeno_zak, bydliste FROM Zakaznik
ORDER BY jmeno_zak;
```

řešení bez duplicit:

```
SELECT DISTINCT jmeno_zak, bydliste FROM Zakaznik
ORDER BY jmeno_zak;
```

vypiš výpůjčky, které byly vráceny do 31.12.2008

řešení:

```
SELECT * FROM Vypujcka
WHERE datum_vrac < '2008-12-31';
```

najdi země, na jejichž filmy jsou nějaké rezervace

řešení:

```
SELECT DISTINCT zeme FROM Film NATURAL JOIN Reservace;
```

nebo

```
SELECT DISTINCT zeme FROM Film JOIN Reservace USING(jmeno_f);
```

nebo

```
SELECT DISTINCT zeme
FROM Film F JOIN Reservace R ON(F.jmeno_f=R.jmeno_f);
```

nebo

```
SELECT DISTINCT zeme FROM Film
WHERE jmeno_f IN (SELECT jmeno_f FROM Reservace);
```

najdi dvojice zákazníků, kteří mají stejnou adresu

řešení:

```
SELECT bydliste, Z1.jmeno_zak, Z2.jmeno_zak
FROM Zakaznik Z1 JOIN Zakaznik Z2 USING(bydliste)
WHERE Z1.RC <> Z2.RC;
```

Poznámka: každá dvojice bude vypsána dvakrát – jednou v pořadí jmen první-druhý, podruhé v pořadí jmen druhý-první. Eliminace těchto věcně duplicitních záznamů není úplně snadná, je možné třeba toto (seřadíme jednoznačně jména zákazníků lexikograficky a pak už lze duplicity eliminovat slovem DISTINCT):

```
SELECT DISTINCT bydliste,
    CASE WHEN Z1.jmeno_zak < Z2.jmeno_zak THEN
        (Z1.jmeno_zak, Z2.jmeno_zak)
```

```

ELSE
    (Z2.jmeno_zak, Z1.jmeno_zak)
END
FROM Zakaznik Z1 JOIN Zakaznik Z2 USING(bydliste)
WHERE Z1.RC <> Z2.RC;

```

najděte zákazníky, kteří si půjčili film Údolí včel

řešení:

```

SELECT DISTINCT Zakaznik.*
FROM Zakaznik NATURAL JOIN Vypujcka NATURAL JOIN Kopie
WHERE jmeno_f = 'Údolí včel';

```

Distinct tam dáváme proto, abychom vyloučil lidi, kteří si tento film půjčili víckrát.

najdi jména zákazníků, kteří mají rezervován nějaký film

řešení:

```

SELECT DISTINCT jmeno_zak
FROM Zakaznik NATURAL JOIN Reservace;

```

Kdyby se dva zákazníci jmenovali stejně, sloučí je to. Aby se to nestalo, buď se musí vynechat DISTINCT (ani pak ovšem nevíme, zda jde o různé lidi nebo opakování jednoho člověka) nebo dát za SELECT ještě klíč zákazníka, tedy rc.

najdi rc a jména zákazníků, kteří nemají rezervován žádný film (zkuste použít EXCEPT, NOT IN, <>ALL, NOT EXISTS, OUTER JOIN + IS NULL, OUTER JOIN + COUNT)

řešení:

```

(SELECT rc, jmeno_zak)
EXCEPT
(SELECT jmeno_zak FROM Zakaznik NATURAL JOIN Reservace);

```

nebo

```

SELECT rc, jmeno_zak
FROM Zakaznik
WHERE rc NOT IN (SELECT rc FROM Reservace);

```

nebo

```

SELECT rc, jmeno_zak
FROM Zakaznik
WHERE rc <>ALL (SELECT rc FROM Reservace);

```

nebo

```

SELECT rc, jmeno_zak
FROM Zakaznik Z
WHERE NOT EXISTS (SELECT * FROM Reservace R WHERE R.rc = Z.rc);

```

nebo

```

SELECT rc, jmeno_zak
FROM Zakaznik LEFT JOIN Reservace USING(rc)
WHERE datum IS NULL;

```

nebo

```

SELECT rc, jmeno_zak
FROM Zakaznik LEFT JOIN Reservace USING(rc)

```

```
GROUP BY rc
HAVING COUNT(jmeno_f) = 0;
```

najdi jména filmů, které jsou právě půjčeny (předpokládejte, že pro dosud nevrácené výpůjčky je datum vrácení nastaveno na NULL)

řešení:

```
SELECT DISTINCT jmeno_f
FROM Kopie NATURAL JOIN Vypujcka
WHERE datum_vrac IS NULL;
```

najděte jména zaměstnanců, kteří někdy půjčili film někomu z Polné

řešení:

```
SELECT jmeno_zam
FROM Zakaznik NATURAL JOIN Vypujcka NATURAL JOIN Zamestnanec
WHERE Zakaznik.bydliste LIKE '%Polná%';
```

vypiš celkovou cenu všech výpůjček Arthura Robinsona v librách (kurs je 1:30)

řešení:

```
SELECT SUM(cena)/30
FROM Vypujcka NATURAL JOIN Zakaznik
WHERE jmeno_zak = 'Arthur Robinson';
```

zjisti celkový počet registrovaných zákazníků

řešení:

```
SELECT COUNT(*)
FROM Zakaznik;
```

kolik je rezervovaných filmů

řešení:

```
SELECT COUNT(DISTINCT jmeno_f)
FROM Reservace;
```

kolik je výpůjček levnějších než 300Kč

řešení:

```
SELECT COUNT(*)
FROM Vypujcka
WHERE cena < 300;
```

jaký je celkový počet výpůjček Jana Nováka

řešení:

```
SELECT COUNT(*)
FROM Vypujcka NATURAL JOIN Zakaznik
WHERE jmeno_zak = 'Jan Novák';
```

najdi pro každého zákazníka (r.č.) počet všech vypůjčených kopií

řešení:

```
SELECT rc, COUNT(datum_vyp)
FROM Vypujcka RIGHT JOIN Zakaznik USING(rc)
GROUP BY rc;
```

Je třeba použít vnější spojení se zákazníkem, abychom dostali i zákazníky, kteří jsou registrovaní, ale nepůjčili si ještě nic.

najdi pro každého zákazníka (r.č.) počet právě vypůjčených kopií (dosud nevrácených)

řešení:

```
SELECT rc, COUNT(datum_vyp) AS 'Počet výpůjček'
FROM Vypujcka RIGHT JOIN Zakaznik USING(rc)
WHERE datum_vrac IS NULL
GROUP BY rc;
```

najdi r.č. zákazníků, kteří mají právě půjčeny víc než dvě kopie

řešení:

```
SELECT rc, COUNT(*)
FROM Vypujcka
WHERE datum_vrac IS NULL
GROUP BY rc
HAVING COUNT(*) > 2;
```

Zde již není vnější spojení se Zákazníkem třeba – chceme pouze rodná čísla s nenulovým počtem výpůjček.

najdi **jména** zákazníků, kteří mají půjčeny víc než dvě kopie (bez a s použitím vnořeného selectu)

řešení:

```
SELECT jmeno_zak
FROM Vypujcka NATURAL JOIN Zakaznik
WHERE datum_vrac IS NULL
GROUP BY rc, jmeno_zak
HAVING COUNT(*) > 2;
```

Výše uvedné řešení musí seskupovat podle dvou sloupců – kdyby seskupovalo pouze podle jména, sloučili by se možní jmenovci; kdyby seskupovalo pouze podle rč, stroj by nedovolil použít sloupec jméno v klauzuli SELECT. Jelikož přiřazení rodných čísel jménům je prosté zobrazení, seskupení podle obou atributů skupinky nijak neovlivní.

nebo

```
SELECT jmeno_zak
FROM Zakaznik NATURAL JOIN
  (SELECT rc
   FROM Vypujcka
   WHERE datum_vrac IS NULL
   GROUP BY rc
   HAVING COUNT(*) > 2) AS X;
```

nebo

```
SELECT jmeno_zak
FROM Zakaznik NATURAL JOIN
  (SELECT rc, COUNT(*) AS pocet
   FROM Vypujcka
   WHERE datum_vrac IS NULL
   GROUP BY rc) AS X
WHERE pocet > 2;
```

nebo

```

SELECT jmeno_zak
FROM Zakaznik
WHERE rc IN
    (SELECT rc
     FROM Vypujcka
     WHERE datum_vrac IS NULL
     GROUP BY rc
     HAVING COUNT(*) > 2);

```

najdi rč a jména zákazníků, kteří nemají rezervován žádný český film

řešení např:

```

SELECT rc, jmeno_zak FROM Zakaznik
WHERE rc NOT IN
    (SELECT rc FROM Reservace NATURAL JOIN Film
     WHERE zeme = 'ČR');

```

najdi rč a jména zákazníků, kteří mají rezervován nebo půjčen japonský film (UNION, OR)

řešení:

```

(SELECT rc, jmeno_zak
 FROM Zakaznik NATURAL JOIN Reservace NATURAL JOIN Film
 WHERE zeme = 'JP')
 UNION
 (SELECT rc, jmeno_zak
  FROM Zakaznik NATURAL JOIN Vypujcka NATURAL JOIN Kopie NATURAL JOIN
   Film
  WHERE zeme = 'JP');

```

nebo

```

SELECT rc, jmeno_zak
FROM Zakaznik
WHERE (rc IN (SELECT rc FROM Reservace NATURAL JOIN Film
              WHERE zeme = 'JP'))
      OR
      (rc IN (SELECT rc FROM Vypujcka NATURAL JOIN Kopie NATURAL
              JOIN Film
              WHERE zeme = 'JP'));

```

najdi filmy, které jsou rezervované a zároveň půjčené (INTERSECT, AND)

řešení:

```

(SELECT DISTINCT jmeno_f FROM Kopie NATURAL JOIN Vypujcka
 WHERE datum_vrac IS NULL)
 INTERSECT
 (SELECT DISTINCT jmeno_f FROM Reservace);

```

nebo

```

SELECT DISTINCT jmeno_f
FROM Film
WHERE (jmeno_f IN (SELECT jmeno_f FROM Kopie NATURAL JOIN Vypujcka
                   WHERE datum_vrac IS NULL)
      AND
      (jmeno_f IN (SELECT jmeno_f FROM Reservace));

```

Povolte u sloupce *cena* prázdné hodnoty a v některých řádcích je použijte místo konkrétní ceny. Vyzkoušejte, jak se na tomto sloupci chovají různé agregační funkce. Navrhněte, jako zajistit vždy správné chování.